

Condensing architectural knowledge from unstructured textual media in agile GSD teams

Gilberto Borrego

Facultad de Ciencias

Universidad Autónoma de Baja California

Ensenada, México

gilberto.borrego@uabc.edu.mx

Abstract— Agile Global Software Development (AGSD) is a reality, since nowadays software products are required to get into the market with more speed than before. This situation has pushed Global Software Development (GSD) companies to adopt lighter ways to develop software (Agile Software Development - ASD) to satisfy market demands. However, AGSD companies have encountered increased technical debt and architectural knowledge (AK) vaporization, mainly because the inherent differences between ASD and GSD, especially in documentation handling. This paper describes a research project in which it is proposed to exploit the AK that is recorded in unstructured textual electronic media (UTEM) generally used in AGSD, in order to decrease the problems of software evolution and maintenance, caused by a lack of adequate AK. Until now, the preliminary results of this research show that this approach could be feasible in AGSD environments.

Keywords— agile global software development; architectural knowledge; unstructured textual media; architectural knowledge vaporization; technical debt, architectural knowledge condensation.

I. CONTEXT

Recently, many companies which apply global software development (GSD), have explored the use of agile software development (ASD) to address the demands of more development speed imposed by the current market conditions [1]. The attention on ASD is due to its strong focus on collaboration, coordination and communication [2], necessary to strengthen the current dynamic.

In principle, ASD and GSD seem to be incompatible. This is inherent to the nature of both paradigms, as ASD was designed for co-located teams, while teams are distributed in GSD. Furthermore, methodological standardization is preferred in GSD, which involves development of plans based on a strict division of tasks and roles, and comprehensive documentation [2]. Moreover, document-based communication is one way to lessen the effect of the four distances inherent in the GSD (linguistic, cultural, temporal and physical) [3]. This kind of communication is contrary to the ASD principles [4], particularly the following: people interactions over processes and tools, working software over a comprehensive documentation, and face-to-face interactions as the best method of conveying information within a development team.

Despite these differences, AGSD companies, i.e. that combine GSD and ASD, have benefits such as strengthening communication, collaboration and trust relationships, reduction of costs, errors and development time, and late requirements management [5]–[8]. However, there are still problems to solve

related to documentation, work distribution, time differences and synchronous communication, support tools, and balancing GSD formality and ASD informality [1], [9].

One of these significant AGSD problems is knowledge management (KM) [10]–[12]. KM is a challenge in AGSD since knowledge transfer is limited [13], despite the communication and collaboration benefits of ASD in GSD. On the one hand, this is presumably because geographical teams distribution in AGSD increases the need for explicit communication (based on documents) [14]. On the other hand, ASD generally results in a reduction of document formality (they are ad-hoc), and on a reduction of the number of documents, as the team decides which documents are needed to understand the project [15]. That is, in AGSD tacit knowledge tends to predominate over explicit knowledge [12], [16].

A relevant kind of knowledge in the software development cycle is that related to software architecture (SA). SA is "the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them" [17]. SA knowledge, or architectural knowledge (AK), also refers to design decisions that led to the current SA [18]. More, AK management (AKM) [19] is also a challenge in AGSD, since ASD reduces the time to write proper documentation, which negatively affects the coding phases [20]. Finally, the most popular agile methods (e.g. Scrum and XP [21], [22]) have lax manners to specify SA, thus SA is often expressed with informal notation [23].

II. MOTIVATION

It is increasingly common that GSD companies adopt ASD. Given the predominance of tacit knowledge in AGSD, these companies commonly have problems related to poor SA documentation, for example: (1) waste of time answering the same questions and finding solutions to problems that had been solved before, (2) loss of knowledge when a project member retires, (3) defects in software evolution and maintenance, (4) lack of visibility for project monitoring and technical solutions, (5) and poorly understood requirements [24], [25].

These problems are related to technical debt, especially in its facet of documentation debt. Technical debt is "all that should belong to a properly developed software system, but which was purposely left out to remain in time or in budget" [26]. Documentation debt refers to documents such as SA design, user manuals, data model, updated requirements specification, etc. [27]. Agile principles suggest to make it "lighter" and "sufficient", however, sometimes the pressure to complete a sprint causes that developers omit documents [26].

On the other hand, agile developers are reluctant about documentation, since they consider it as a secondary and non-creative activity [28].

Both the lack of AK, as increasing technical debt, favor AK vaporization, i.e., loss of AK over time by not having it properly documented [29]. When developers do not clearly understand the SA of a project, a greater effort is required to evolve it and maintain it, since they ignore the design decisions which led to the current software structure [30]. Even, between 40% and 60% of maintenance time is dedicated to understand a software when SA documentation is inadequate [31].

Moreover, the effects of AK vaporization in ASD are not exclusive to software maintenance. For example, when a person withdraws from an ongoing project, and there is inadequate SA documentation, AK is vaporized by the absence of a team member. In the former case, AK is not completely vaporized, thanks to the shared source code ownership of ASD. However, it is complicated to extract tacit elements (e.g. design decisions) by code examination only. Moreover, while in ASD AK vaporization is catered with frequent interaction among developers, in AGSD the four distances cause less efficiency in AK sharing, as opportunities for casual interaction [32] and informal awareness [33] are missed. Once the AK has been vaporized, the only alternative to recover it, is analyzing the source code. This task is time consuming, and the knowledge obtained in this manner is generally unstructured, incomplete, and inconsistent [34], which does not guarantee that the software evolve as it was planned at designing time.

III. PROBLEM STATEMENT

The predominance of tacit knowledge over explicit knowledge in AGSD converts AKM into a challenge in AGSD companies. This leads into an increased technical debt and AK vaporization, which also leads into problems in software evolution and maintenance.

To the best of our knowledge, there are a few works that address AKM in AGSD. Babar [35] found that agile developers (distributed or not) use Wikis to share AK with informal formats. However, he reported that AK is difficult to find in Wikis, due to informal formats. Also, there are efforts to design lightweight methods to integrate AKM in AGSD [36], [37], however field tests are not reported for these methods. Clerc et al. [16] validated AKM practices of GSD in AGSD projects, finding that personalization techniques predominate over centralization. Finally, Urrego et al. [38] created a tablet application to capture AK during design meetings between local and remote members, but it requires more field testing.

Thus, the literature shows that there is a lack of solutions to reduce technical debt and AK vaporization in AGSD environments, to ease software evolution and maintenance, impacting to the least extent possible the beneficial aspects of ASD and GSD.

IV. SOLUTION APPROACH

In AGSD communication between local and remote team members is carried out through phone calls, instant messaging (IM), email and video calls [16]. However, developers prefer text media, because they reduce communication problems caused by language differences [39]. These means (which we

will call unstructured textual electronic media – UTEM), leave a trace of interactions (logs), and given the agile nature of the processes, there will be AK that will not be probably recorded anywhere else. Even, comments on repository hosting services (e.g. Github) have been a source of AK, in the absence of formal documentation [40]. Hence, UTEM logs can be valuable to complement and contextualize informal and ad-hoc documents used in AGSD and thus reduce technical debt and AK vaporization, by “condensing” the AK in them.

V. GAPS

To the best of our knowledge, there is few evidence in the literature of the use of UTEM as a source of AK. Juarez et al. [41] found AK in Facebook entries, in scholar software projects, where Facebook was used as an interaction mean. Furthermore, Clerc et al. [16] reported that UTEM are used to share AK in AGSD projects. However, these works do not specify what kind of AK is shared through UTEM (e.g. detailed design, requirements, overall architecture, etc.). Also, machine learning techniques have been used to exploit informal sources. In [42], emails were analyzed to relate them to user stories depending of the email context. In [43], entries of Q&A sites were analyzed to extract features requests. However, in both projects accuracy was around 70%, which would not be acceptable for many development environments. Moreover, the latter works do not consider the IM preference of developers to share AK in AGSD [39].

VI. RESEARCH QUESTIONS

Once stated the research problem and solution approach, the main research question is:

- What kind of support is needed to perform AKM in AGSD leveraging UTEM traces, maintaining a balance between the preference of tacit knowledge of ASD and the need for explicit knowledge of GSD?

At the same time, the secondary questions are the following:

- Which aspects of AK are shared through UTEM in AGSD environments?
- How can architectural artifacts be integrated with AK shared through UTEM in AGSD environments?
- How to technologically support AKM leveraging the AK shared through UTEM in AGSD environments?
- Which is the impact of implementing the AKM in AGSD based on the use of AK shared through UTEM?

VII. METHOD

The method proposed to guide this research is briefly explained below.

Understanding. This phase consists of a literature review of AKM in AGSD, and an on-site observational study to understand the articulation of AK in UTEM in an AGSD company. Then, the results of this study are compared with the practices of other AGSD companies through focus group sessions. The results of both studies are brought together to obtain an understanding of how AKM is done in those AGSD companies.

Elaborating. In this phase a two-iteration loop is started, in which the technological support solution for AKM in

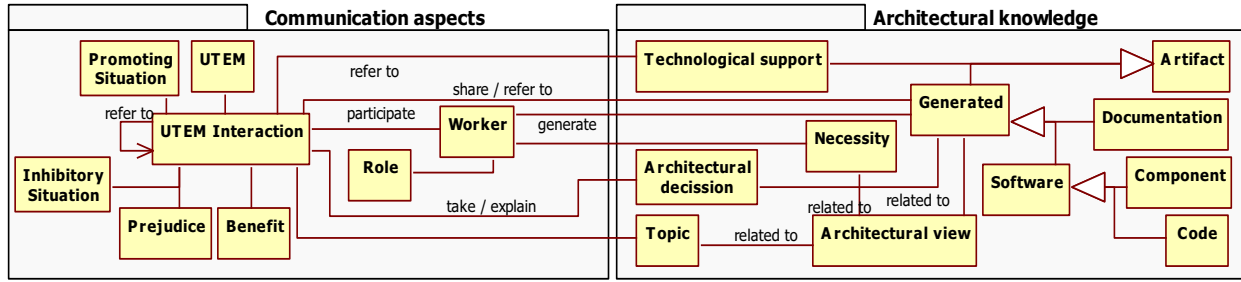


Fig. 1. Ontology that represents the involved aspects in AK articulation through UTEM in AGSD teams (all relations without label are of type “have”)

AGSD is refined, applying user centered design. In the first iteration, based on the results of the understanding phase, the first version of the technological support is designed and implemented through a low-fidelity prototype. This prototype will be evaluated in terms of perceived usability and intended use. In the second iteration, the results of the latter evaluation are used as a basis to improve the next version of the technological support. This second evaluation will be conducted in an AGSD company, and the usability and intended use of this technological support will be evaluated, as well as the performance of participants.

Conclusion. In this final phase the results of the second iteration of the elaborating phase are analyzed in order to obtain the final conclusions of this study.

VIII. INITIAL RESULTS

As first results of the on-site observational study, we captured UTEM (IM, email and forums) interactions related to AK. We analyzed them using the architectural model 4+1 views and we obtained that issues concerning to scenarios and implementation views are prominent in UTEM interactions. This behavior is aligned with the usual behavior in ASD, i.e., the analysis and design phases are diluted, because the problems are generally understood by experimenting in code [34]. In addition, as a result of overall understanding of the AK articulation in AGSD, we obtained an ontological representation (see Fig. 1), where two areas were identified: communication aspects and AK aspects. Finally, we obtained a list of AK topics shared by UTEM, which was evaluated by the participant developers in terms of incidence and importance. We found that the perceived importance of the evaluated topics is significantly higher than its perceived incidence. That is, a topic is considered as valuable by AGSD developers regardless of how often it is treated through UTEM.

IX. CONCLUSIONS AND FUTURE WORK

The adoption of ASD in GSD is bringing new challenges, where AKM is included, mainly due to a difference of perspectives between ASD and GSD, regarding software documentation. Agile AKM practices predominate in AGSD, which brings increased technical debt and AK vaporization. The proposed research addresses this problem by “condensing” the AK shared through UTEM, i.e., the exploitation of articulated AK in UTEM, since there is evidence that these means are preferred by developers in AGSD environments.

Our first results provide preliminary evidence about this way of addressing the problem, which could be helpful. Future

work will be to design, implement and evaluate the first version of the technological support to condense AK. One feature that so far we see as mandatory, is that the technological support should be naturally integrated into the work dynamics in AGSD, so that team members do not see it as an extra workload. For this reason we propose the use of tagging during UTEM interactions, aiming at relating them to our AK ontology. Since these interactions will be captured, team members will be able to search AK in terms of the ontology tags. We chose the tagging technique because nowadays it is unobtrusively and successfully used in software development [44].

ACKNOWLEDGMENT

I thank my thesis directors PhD. Alberto L. Morán and PhD. Ramón R. Palacio Cinco, as well as the rest of my thesis committee: PhD. Oscar M. Rodríguez Elías, PhD Marcela D. Rodríguez Urrea and PhD. Eloísa del Carmen García Canseco.

REFERENCES

- [1] B. Ramesh, L. Cao, K. Mohan, and P. Xu, “Can distributed software development be agile?,” *Commun. ACM*, vol. 49, no. 10, p. 41, Oct. 2006.
- [2] D. Šmite, P. J. Ågerfalk, and N. B. Moe, “Fundamentals of Agile Distributed Software Development,” in *Agility Across Time and Space: Implementing Agile Methods in Global Software Projects*, 2010, pp. 1–341.
- [3] A. Vizcaíno, F. García, M. Piattini, and P. Garrido, “El desarrollo global del software,” in *Desarrollo Global de Software*, A. Vizcaíno, F. García, and M. Piattini, Eds. Ra-ma, 2014, pp. 37–58.
- [4] M. Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, “The Agile Manifesto.” 2001.
- [5] S. Jalali and C. Wohlin, “Agile practices in global software engineering—A systematic map,” in ... , *2010 5th IEEE International Conference on*, 2010, pp. 45 – 54.
- [6] H. Holmström, B. Fitzgerald, P. J. Ågerfalk, and E. Ó. Conchúir, “Agile Practices Reduce Distance in Global Software Development,” *Inf. Syst. Manag.*, vol. 23, no. 3, pp. 7–18, 2006.
- [7] Y. Li and A. Maedche, “Formulating Effective Coordination Strategies in Agile Global Software Development Teams,” in *Proceedings of the International Conference on Information Systems*, 2012.
- [8] S. V. Shrivastava and H. Date, “Distributed Agile Software Development: A Review,” *J. Comput. Sci. Eng.*, vol. 1, no. 1, pp. 10–17, 2010.
- [9] S. V. Shrivastava and H. Date, “Distributed Agile Software

- Development: A Review,” vol. 1, no. 1, pp. 10–17, 2010.
- [10] S. Dorairaj, J. Noble, and P. Malik, “Knowledge Management in Distributed Agile Software Development,” in *2012 Agile Conference*, 2012, pp. 64–73.
- [11] M. Jiménez, M. Piattini, and A. Vizcaino, “Challenges and Improvements in Distributed Software Development: A Systematic Review,” *Adv. Softw. Eng.*, vol. 2009, no. 710971, p. 14, 2009.
- [12] A. R. Yanzer Cabral, M. B. Ribeiro, and R. P. Noll, “Knowledge Management in Agile Software Projects: A Systematic Review,” *J. Inf. Knowl. Manag.*, vol. 13, no. 01, p. 1450010, 2014.
- [13] N. Kamaruddin, N. Arshad, and A. Mohamed, “Comparison of drivers between global software development and agile global software development: A SURVEY,” in *Computer and Mathematical Sciences Graduates National Colloquium 2013, At Faculty of Computer and Mathematical Sciences, UiTM Shah Ala*, 2012.
- [14] P. Kruchten, “Contextualizing agile software development,” *J. Softw. Evol. Process*, vol. 25, no. 4, pp. 351–361, 2013.
- [15] D. Turk, R. France, and B. Rumpe, “Limitations of agile software processes,” in *Third International Conference on Extreme Programming and Flexible Processes in Software Engineering*, 2002, pp. 43–46.
- [16] V. Clerc, P. Lago, and H. Van Vliet, “Architectural Knowledge Management Practices in Agile Global Software Development,” in *2011 IEEE Sixth International Conference on Global Software Engineering Workshop*, 2011, pp. 1–8.
- [17] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 2nd ed. Addison-Wesley Professional, 2003.
- [18] M. Babar, T. Dingsøyr, P. Lago, and H. van Vliet, *Software architecture knowledge management: theory and practice*. Springer Berlin Heidelberg, 2009.
- [19] H. Van Vliet, “Software architecture knowledge management,” in *Software Engineering, 2008. ASWEC 2008. 19th ...*, 2008, pp. 24–31.
- [20] R. Noordeloos and C. Manteli, “From RUP to Scrum in Global Software Development: A Case Study,” 2012.
- [21] I. Ghani and M. Bello, “Agile adoption in IT organizations,” *KSII Trans. Internet Inf. Syst.*, vol. 9, no. 8, pp. 3231–3248, 2015.
- [22] A. M. M. Hamed and H. Abushama, “Popular agile approaches in software development: Review and analysis,” in *2013 International Conference on Computing, Electrical and Electronic Engineering (Iccee)*, 2013, pp. 160–166.
- [23] R. Hoda, J. Noble, and S. Marshall, “How much is just enough?,” in *Proceedings of the 15th European Conference on Pattern Languages of Programs - EuroPLOP '10*, 2010, no. EuroPLOP, p. 13.
- [24] H. Holz and F. Maurer, “Knowledge management support for distributed agile software processes,” *Adv. Learn. Softw. Organ.*, vol. 2640, pp. 60–80, 2003.
- [25] N. Uikey, U. Suman, and A. Ramani, “A Documented Approach in Agile Software Development,” *Int. J. Softw. ...*, vol. 2, no. 2, pp. 13–22, 2011.
- [26] H. M. Sneed, “Dealing with Technical Debt in agile development projects,” *Lect. Notes Bus. Inf. Process.*, vol. 166 LNBIP, pp. 48–62, 2014.
- [27] E. Tom, A. Aurum, and R. Vidgen, “An exploration of technical debt,” *J. Syst. Softw.*, vol. 86, no. 6, pp. 1498–1516, 2013.
- [28] T. Clear, “Documentation and Agile Methods: Striking a Balance,” *SIGCSE Bull.*, vol. 35, no. 2, pp. 12–13, Jun. 2003.
- [29] J. Bosch, “Software Architecture: The Next Step,” in *EWSA*, 2004, vol. 3047, pp. 194–199.
- [30] M. Galster and M. A. Babar, “Empirical Study of Architectural Knowledge Management Practices,” *2014 IEEE/IFIP Conf. Softw. Archit.*, pp. 239–242, Apr. 2014.
- [31] S. L. Pfleeger, *Software Engineering: Theory and Practice*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2001.
- [32] E. Isaacs, S. Whittaker, D. Frohlich, and B. O’Conaill, “Informal communication re-examined: New functions for video in supporting opportunistic encounters,” *... -Mediated Commun.*, pp. 1–30, 1997.
- [33] C. Gutwin, S. Greenberg, and M. Roseman, “Workspace Awareness in Real-Time Distributed Groupware: Framework, Widgets, and Evaluation,” in *People and Computers XI*, M. A. Sasse, R. J. Cunningham, and R. L. Winder, Eds. London: Springer London, 1996, pp. 281–298.
- [34] B. Selic, “Agile Documentation, Anyone?,” *IEEE Softw.*, vol. 26, no. 6, pp. 11–12, 2009.
- [35] M. a. Babar, “An exploratory study of architectural practices and challenges in using agile software development approaches,” in *2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture*, 2009, pp. 81–90.
- [36] I. Hadar, S. Sherman, E. Hadar, and J. J. Harrison, “Less is more: Architecture documentation for agile development,” *2013 6th Int. Work. Coop. Hum. Asp. Softw. Eng. CHASE 2013 - Proc.*, pp. 121–124, 2013.
- [37] V. Eloranta and K. Koskimies, “Lightweight Architecture Knowledge Management for Agile Software Development,” in *Agile Software Architecture: Aligning Agile Processes and Software Architectures*, Elsevier Science, 2013, pp. 189–211.
- [38] J. Urrego, R. Munoz, M. Mercado, and D. Correal, “Archinotes: A global agile architecture design approach,” *Lect. Notes Bus. Inf. Process.*, vol. 179 LNBIP, pp. 302–311, 2014.
- [39] H.-C. Estler, M. Nordio, C. A. Furia, B. Meyer, and J. Schneider, “Agile vs. Structured Distributed Software Development: A Case Study,” in *Proceedings of the IEEE International Conference on Global Software Engineering*, 2012.
- [40] J. Sheoran, K. Blincoe, E. Kalliamvakou, D. Damian, and J. Ell, “Understanding ‘Watchers’ on GitHub,” in *Proceedings of the 11th Working Conference on Mining Software Repositories*, 2014, pp. 336–339.
- [41] R. Juárez, R. Pimienta, and V. Ocegueda, “Using Social Networks for Integrating a Tacit Knowledge Repository to Support,” in *Integrated Uncertainty in Knowledge Modelling and Decision Making*, Beijing, China, 2013, pp. 167–179.
- [42] S. M. Sohan, M. M. Richter, and F. Maurer, “Auto-tagging emails with user stories using project context,” *Lect. Notes Bus. Inf. Process.*, vol. 48 LNBIP, pp. 103–116, 2010.
- [43] M. Xiao, G. Yin, T. Wang, C. Yang, and M. Chen, “Requirement Acquisition from Social Q&A Sites,” in *Requirements Engineering in the Big Data Era*, vol. 558, no. February 2016, L. Liu and M. Aoyama, Eds. Springer Berlin Heidelberg, 2015, pp. 64–74.
- [44] R. Giuffrida and Y. Dittrich, “Empirical studies on the use of social software in global software development-A systematic mapping study,” *Inf. Softw. Technol.*, vol. 55, no. 7, pp. 1143–1164, 2013.